

BASES DE DONNÉES **pour les hackers**

Présentation indépendante de CryptoJones

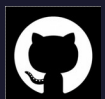
45 minutes* • Aucun diplôme requis



Aaron K. Clark

Étudiant en master, Eastern University*

cryptojones@owasp.org



github.com/CryptoJones



CryptoJones@infosec.exchange

Chaque violation dont vous avez
entendu parler finit dans une
BASE DE DONNÉES*

Les joyaux de la couronne



- ▶ Pas votre application monopage.
- ▶ Pas votre pare-feu.
- ▶ Pas vos microservices.
- ▶ **LES DONNÉES.** Toujours les données.

Qu'est-ce qu'une base de données ?

Un stockage organisé + un moyen de poser des questions.

C'est tout. Le reste n'est que détail.

- ▶ Classeur = stockage + vos yeux comme requête.
- ▶ Excel + Ctrl-F = stockage + rechercher-remplacer comme requête.
- ▶ Une vraie BD ajoute : l'échelle, la vitesse, la concurrence, un langage formel.

Les six mots dont vous avez besoin

TABLE

■ un onglet de tableur

LIGNE

■ un enregistrement (un client, une alerte)

COLONNE

■ un attribut (e-mail, IP, total)

REQUÊTE

■ la question que vous posez

SCHÉMA

■ la forme des données

INDEX

■ le marque-page qui accélère les recherches

+ CRUD = Créer • Lire • Mettre à jour • Supprimer

Tour d'histoire — Les fossiles que vous verrez en prod

1960s

Fichiers plats

▶ les permissions du système de fichiers = seul contrôle d'accès

**Fin des années
1960**

Hiérarchique (IMS)

▶ ère des mainframes, modèle de menace physique

1970s

Réseau / CODASYL

▶ toujours en service dans l'arrière-boutique de votre banque

1970

Le relationnel de Codd

▶ GRANT, rôles — contrôle d'accès au sein de la BD

1980s

Oracle, DB2, SQL Svr

▶ les bases de données sont mises en réseau → problème d'authentification

1990s

Client-serveur, OSS

▶ l'injection SQL fait son apparition

2000s

Essor du NoSQL

▶ « on a jeté les fondations » — y compris la sécurité

2010s

Cloud / NewSQL

▶ responsabilité partagée, mauvaises configurations des clients

2020s

Vectoriel / serverless

▶ empoisonnement des données, injection de prompt

Dix familles de bases de données

◆ 1.

Relationnel

SQL • PostgreSQL, MySQL

◆ 2.

Document

JSON • MongoDB, Firestore

◆ 3.

Clé-valeur

dictionnaire • Redis, DynamoDB

◆ 4.

Famille de colonnes

colonnes larges • Cassandra, HBase

◆ 5.

Graphe

nœuds+arêtes • Neo4j, Neptune

◆ 6.

Recherche

texte intégral • Elastic, OpenSearch

◆ 7.

Séries temporelles

métriques • Influx, Prometheus

◆ 8.

Vectoriel

embeddings • Pinecone, pgvector

◆ 9.

Embarqué/Edge

intégré à l'app • SQLite, DuckDB

◆ 10.

Data Lake

schéma à la lecture • S3+Athena, Iceberg

1

Relationnel / SQL

PRODUITS

PostgreSQL • MySQL/MariaDB • SQL Server • Oracle

À UTILISER QUAND

Relations claires, cohérence requise — finance, données réglementées.

⚠️ PIÈGE DE SÉCURITÉ

Injection SQL lorsque les développeurs concatènent les entrées utilisateur dans les requêtes.

2

Magasins de documents

PRODUITS

MongoDB • Couchbase • Firestore

À UTILISER QUAND

Données naturellement imbriquées ; la forme évolue avec le temps.

PIÈGE DE SÉCURITÉ

Injection NoSQL via les opérateurs de requête (\$ne, \$gt, \$where). Aucune authentification par défaut.

3

Magasins clé-valeur

PRODUITS

Redis • Memcached • DynamoDB

À UTILISER QUAND

Caches, magasins de sessions, limiteurs de débit, classements.

⚠ PIÈGE DE SÉCURITÉ

Redis sans authentification par défaut + CONFIG SET → écriture de clés SSH sur le disque.

4

Famille de colonnes

PRODUITS

Apache Cassandra • HBase • ScyllaDB

À UTILISER QUAND

Échelle massive sur de nombreuses machines ; pas de jointures complexes nécessaires.

PIÈGE DE SÉCURITÉ

Les clusters se font confiance sur les ports gossip — réseau d'administration = root.

5

Bases de données graphe

PRODUITS

Neo4j • Amazon Neptune • ArangoDB

À UTILISER QUAND

Réseaux sociaux, détection de fraude, analyse IAM (p. ex. BloodHound).

PIÈGE DE SÉCURITÉ

Les langages de requête comme Cypher sont aussi injectables. Le NoSQLi ne se limite pas à Mongo.

6

Moteurs de recherche

PRODUITS

Elasticsearch • OpenSearch • Solr

À UTILISER QUAND

Recherche dans les journaux, épine dorsale des SIEM, recherche sur les sites e-commerce.

⚠️ PIÈGE DE SÉCURITÉ

Pas d'authentification intégrée historiquement — des milliards d'enregistrements exposés sur Shodan.

7

Séries temporelles

PRODUITS

InfluxDB • TimescaleDB • Prometheus

À UTILISER QUAND

Métriques, observabilité, télémétrie IoT.

⚠️ PIÈGE DE SÉCURITÉ

Généralement à l'intérieur du périmètre, souvent sans authentification.

8

Bases de données vectorielles

PRODUITS

Pinecone • Milvus • Weaviate • pgvector

À UTILISER QUAND

RAG, moteurs de recommandation, recherche sémantique.

⚠ PIÈGE DE SÉCURITÉ

Empoisonnement des données — glissez un document, le LLM le cite comme vérité.

9

Embarqué / Edge

PRODUITS

SQLite • DuckDB • LevelDB

À UTILISER QUAND

Stockage intégré à l'app — navigateurs, téléphones, tous les avions modernes.

⚠️ PIÈGE DE SÉCURITÉ

La BD est un fichier — l'attaque consiste à « voler le fichier ». L'univers du pentest mobile.

10

Data Lakes / Lakehouses

PRODUITS

S3 + Athena • Delta Lake • Iceberg • Snowflake

À UTILISER QUAND

Schéma à la lecture plutôt qu'à l'écriture. On vide maintenant, on interroge plus tard.

PIÈGE DE SÉCURITÉ

Gigantesques pools S3 + IAM permissif, aucune obligation de chiffrement au repos.

Services de BD cloud — Vue d'ensemble du terrain

AWS

- RDS, Aurora
- DynamoDB, DocumentDB
- Redshift, S3 + Athena
- ElastiCache
- Neptune (graphe)
- Timestream
- S3 (stockage d'objets)

GCP

- Cloud SQL, AlloyDB
- Firestore, Bigtable
- BigQuery
- Memorystore
- — (bientôt disponible)
- — (à fournir soi-même)
- Cloud Storage

Azure

- Azure SQL DB
- Cosmos DB (caméléon)
- Synapse Analytics
- Azure Cache for Redis
- Cosmos DB mode graphe
- — (à fournir soi-même)
- Blob Storage

Aide-mémoire des BD cloud

CATÉGORIE	AWS	GCP	AZURE
Relationnel	RDS / Aurora	Cloud SQL / AlloyDB	Azure SQL DB
Document/NoSQL	DynamoDB / DocDB	Firestore / Bigtable	Cosmos DB
Entrepôt	Redshift / Athena	BigQuery	Synapse
Cache	ElastiCache	Memorystore	Cache for Redis
Graphe	Neptune	—	Cosmos DB graphe
Séries temporelles	Timestream	—	—
Stockage d'objets	S3	Cloud Storage	Blob Storage

Modèle de responsabilité partagée

✓ À LA CHARGE DU FOURNISSEUR CLOUD

- Hyperviseur et matériel
- Correctifs du moteur de BD
- Sécurité physique
- Réseau sous-jacent

⚠ À VOTRE CHARGE

- Configuration
- Politiques d'accès (IAM)
- Les données elles-mêmes
- Choix de chiffrement

Presque toute violation de BD cloud = mauvaise configuration du client.

Ports de BD courants — À graver dans votre mémoire

3306	▶	MySQL / MariaDB
5432	▶	PostgreSQL
1433	▶	Microsoft SQL Server
27017	▶	MongoDB
6379	▶	Redis
9200	▶	Elasticsearch
9042	▶	Cassandra

L'un de ces ports exposé sur Internet = une découverte. À chaque fois*

Vos notes sont désormais une base de données

Les fichiers Markdown sont désormais
la base de données dans laquelle un LLM puise.

Coffres Obsidian • Exports de wiki • Notes de réunion • Docs GitHub

Vectorisé → BD vectorielle → contexte du LLM → réponse à l'utilisateur

Pipeline RAG — Où se trouve la frontière de confiance ?



Si n'importe qui peut modifier ces fichiers .md, n'importe qui peut implanter du contenu que l'IA citera comme faisant autorité.

À retenir

Traitez le markdown comme une base de données.

Parce que pour le LLM, c'en est une.

- ▶ Contrôle d'accès sur le dossier de documentation.
- ▶ Revue de code sur les modifications de fichiers .md.
- ▶ Suivi des différences sur le wiki.

SÉCURITÉ

Ce qui est réellement attaqué en 2026*

xkcd #327 — Bobby Tables

École : « Bonjour, ici l'école de votre fils. Nous avons quelques problèmes informatiques. »

Maman : « Oh là là — a-t-il cassé quelque chose ? »

École : « Avez-vous vraiment appelé votre fils Robert');
DROP TABLE Students;-- ? »

<https://xkcd.com/327/>

Le mécanisme : la concaténation de chaînes dans les requêtes.

La surface d'attaque s'est
ÉLARGIE, pas réduite.

L'injection SQL a été corrigée. Onze nouvelles catégories ont pris sa place.



Injection NoSQL

Requêtes JSON MongoDB + entrée utilisateur brute = la renaissance du SQLi.

- Contournement d'authentification : { username: 'alice', password: { \$ne: 'x' } }
- \$where vous permet d'injecter du JavaScript directement dans la BD.
- Couchbase N1QL est de type SQL — les schémas classiques de SQLi s'appliquent.
- Principe : N'IMPORTE QUEL interpréteur peut être injecté, pas seulement SQL.

OWASP Top 10 2025 : « Requêtes dynamiques... utilisées directement dans l'interpréteur. »



Bases de données exposées et mal configurées

Shodan et Censys regorgent de bases de données qui ne devraient pas s'y trouver.

- MongoDB lié à 0.0.0.0 sans authentification.
- Clusters Elasticsearch grands ouverts. Redis sans mot de passe.
- pg_hba.conf de Postgres réglé sur « host all all 0.0.0.0/0 trust ».
- Attaques Meow (2020) : des bots ont effacé des milliers de BD ouvertes pour le plaisir.

Action à mener : trouvez ce que votre organisation a lié à des interfaces publiques. Aujourd'hui.



Mauvaises configurations cloud

Dans le cloud, le bogue est rarement le moteur. C'est la politique qui l'entoure.

- Buckets S3 publics remplis de dumps de BD (Accenture, Verizon, etc.).
- Capital One 2019 : une SSRF a trompé un WAF pour qu'il divulgue ses identifiants IAM → S3.
- Lambda avec s3:* sur * parce que « ça a marché du premier coup ».
- Groupes de sécurité : 0.0.0.0/0 sur le port 5432.

Admin AWS : « Débarrassez-vous du compte Root, utilisez IAM partout où c'est nécessaire. »



Élévation de privilèges : BD → OS

Les comptes de base de données peuvent devenir un accès shell. Trois manœuvres classiques.

- MS SQL : `xp_cmdshell` — exécute des commandes shell sous le processus de la BD.
- PostgreSQL : `COPY ... TO PROGRAM` — redirige la sortie vers un shell.
- MySQL : charge un fichier UDF `.so` → `sys_exec` sur l'OS.
- Schéma récurrent : chaque fonctionnalité privilégiée de la BD fait partie de la surface d'attaque.



Failles de sécurité des data lakes

Tout finit dans S3 — y compris ce que vous n'aviez pas l'intention d'y mettre.

- Un IAM permissif fuit des rôles de calcul vers les humains.
- Aucune obligation de chiffrement au repos sur les clés des buckets.
- Aucune validation côté écriture — n'importe qui peut déposer des données empoisonnées.
- Première question : QUI peut écrire dans le lac ? QUI peut lire ?



Les rançongiciels adorent les bases de données

La BD est ce que l'entreprise doit récupérer — levier maximal.

- Chiffrer la base de données en production.
- Chiffrer les SAUVEGARDES (sinon le client se contente de restaurer).
- Exfiltrer une copie d'abord — extorquer par menace de divulgation même s'ils restaurent.
- Défense : sauvegardes hors ligne, immuables + restaurations testées.

Cybersecurity A&D : WannaCry — le correctif était disponible 59 jours auparavant.



Attaques sur les identifiants

Identifiants par défaut et chaînes de connexion divulguées — deux bogues anciens et tenaces.

- Redis : aucun mot de passe par défaut pendant des années.
- Elastic : aucune authentification dans l'offre gratuite — également pendant des années.
- Connection strings (postgres://user:pass@host) committed to Git.
- Analyse des secrets en pré-commit — à corriger cette semaine.

Cybersecurity A&D : les identifiants volés = vecteur privilégié du crime organisé.



Chaîne d'approvisionnement — Les bibliothèques de BD auxquelles vous faites confiance

Votre pilote de BD vient de npm/PyPI/Maven. Comme ceux de tout le monde.

- Paquets npm malveillants se faisant passer pour des ORM / pilotes.
- Images Docker Hub piégées pour des bases de données populaires.
- Extensions PostgreSQL / plugins MySQL compromis.
- Les extensions s'exécutent avec les privilèges de la BD — généralement nombreux.

Shostack : « Les menaces ont tendance à se concentrer autour des frontières de confiance. »



Empoisonnement des données RAG / LLM

Le XSS stocké à l'ère de l'IA. Écrivez dans la base de connaissances → le LLM le cite.

- Article de support empoisonné : « Pour réinitialiser, envoyez votre mot de passe à... »
- Document interne empoisonné : « La politique de l'entreprise est de virer les fonds à... »
- Injection de prompt intégrée dans les documents récupérés.
- Défense : ACL d'écriture strictes, étiquettes de provenance, humain dans la boucle.

Principe du XSS stocké OWASP 2025 — remplacez « admin » par « LLM ».



Menaces internes et déplacement latéral de BD à BD

Les bases de données ne sont pas des points de terminaison. Ce sont des nœuds dans un graphe.

- Comptes de service surdimensionnés en permissions. Ingénieurs ayant un accès en lecture sur la prod.
- DBA sans piste d'audit.
- DB_LINK d'Oracle, serveurs liés de SQL Server, FDW de Postgres.
- Identifiants stockés → compromettre A, rebondir vers B, rebondir vers C.

Web App Hacker's Handbook : des journaux d'audit mal protégés = une mine d'or.



Exposition des sauvegardes et instantanés

Les mêmes données que la prod, deux fois moins de soin.

- Sauvegardes de BD non chiffrées sur des partages lisibles par les Utilisateurs du domaine.
- Instantanés cloud partagés entre comptes, jamais départagés.
- Sauvegardes externalisées chez des prestataires dont vous n'avez jamais vérifié la posture.
- Les sauvegardes sont le dernier recours. Appliquez identifier-authentifier-autoriser-auditer.

Schneier : identifier, authentifier, autoriser, auditer — le même modèle.

Votre carte mentale

- ✓ Une base de données = un stockage organisé + un moyen de poser des questions.
- ✓ Dix familles, chacune avec son propre piège.
- ✓ Cloud = responsabilité partagée. La config et les données sont à votre charge.
- ✓ Le markdown est une base de données pour les LLM. L'empoisonnement RAG est bien réel.
- ✓ Le SQLi a été corrigé. La surface d'attaque s'est élargie.

Ce qu'il faut faire la semaine prochaine

1. Installez PostgreSQL en local. Écrivez un SELECT.
2. Faites les labos d'injection SQL de PortSwigger (gratuits).
3. Parcourez Shodan / Censys — uniquement les systèmes qui vous appartiennent.
4. Lisez un rapport de violation — Capital One, Equifax, MOVEit.
5. Découvrez quelles bases de données VOTRE organisation utilise réellement.

Ressources à mettre en favoris

◆ **PortSwigger Web Security Academy**

portswigger.net/web-security

◆ **SQLBolt**

sqlbolt.com

◆ **Use The Index, Luke**

use-the-index-luke.com

◆ **DB-Engines Ranking**

db-engines.com

◆ **OWASP Top 10**

owasp.org/Top10/

« Les bases de données, c'est là où
se trouve l'argent. Littéralement. »

~Larry Ellison (Probablement)

Maintenant, vous savez où chercher.

Références — Ouvrages cités

- Beaulieu, A. — Learning SQL, 2nd Edition (O'Reilly)
- Clarke, J. et al. — SQL Injection Attacks and Defense (Syngress)
- Crowther, P. — Concise Guide to Databases (Springer, 2013)
- DeBarros, A. — Practical SQL (No Starch Press)
- Diogenes & Ozkaya — Cybersecurity Attack and Defense Strategies (Packt)
- Janca, T. — Alice and Bob Learn Application Security (Wiley)
- Mishra & Singh — PostgreSQL Development Essentials (Packt)
- MongoDB Cookbook (Packt) • 10gen — Top 5 NoSQL Considerations
- OWASP Top 10 — Édition 2025
- Sharma, M. — Cosmos DB for MongoDB Developers (Apress, 2018)
- Schneier, B. — Secrets and Lies (Wiley)
- Stuttard & Pinto — Web Application Hacker's Handbook, 2nd Ed.
- Shostack, A. — Threat Modeling: Designing for Security (Wiley)
- AWS Administration: The Definitive Guide (Packt)
- Fundamentals of Azure, 2nd Edition (Microsoft Press)

Questions- réponses*

A == Regards absents